Chapter 2: SQL in Python

In the SQLite shell, we typed a statement and inspected the result. Now a Python program will do that work. The Structured Query Language (SQL) statements are familiar, but we have another set of decisions to make: where the values come from, how to use the returned records, and what the program should do when an operation fails.

The example is db-example.py in `topic-02-sql-in-python`. It works with the same pet table as the introductory SQL session. This chapter follows the script, with some line breaks and indentation adjusted for readability. Examples that extend the script are identified separately.

Running the Example

From the topic directory, run:

```
python3 db-example.py --db practice.db
```

The script drops and recreates the `pet` table in the selected database. Use a practice filename. Running it again resets the pets rather than continuing from the previous run's state.

That makes the demonstration repeatable, but it is not how we would normally start an application that needs to preserve existing records. We will separate setup from ordinary application operations as the course develops.

Without `--db`, the script uses `pets.db` in the current working directory.

Imports and Command-Line Arguments

The script begins:

```
import argparse
import sqlite3
from pprint import pprint

ap = argparse.ArgumentParser()
ap.add_argument("--db", default="pets.db")
args = ap.parse_args()
```

`sqlite3` provides the database connection. `argparse` reads the program's command-line arguments. `pprint` prints Python values in a readable form; it does not perform database operations.

The `--db` option lets us change the database filename without editing the source. After parsing, `args.db` holds either the supplied filename or the default. In the command above, it is the string `"practice.db"`.

This is already a small example of application data becoming input to a database operation. Python obtains a value, stores it in an object, and passes it to another part of the program.

Opening a Connection

```
connection = sqlite3.connect(args.db)

print("succeeded in making connection.")
```

The connection is the Python object through which the script works with the database. It remains in memory while the program runs; the database file can outlast that process.

A relative filename still depends on the current working directory. The Python script's location does not automatically become the base directory for `practice.db`.

The success message runs only if `connect()` returns. It confirms that the connection opened, not that all later operations will succeed.

Rebuilding the Table

The first database operation clears the previous demonstration:

```
connection.execute("drop table if exists pet")
connection.commit()
```

`execute()` sends SQL to the database. Here, `IF EXISTS` allows the script to continue when there is no `pet` table yet. If the table does exist, dropping it removes its records along with its definition.

The script then creates the table:

```
connection.execute("""
    create table pet
    (
        id integer primary key autoincrement,
        name text not null,
        kind text not null,
        age integer,
        food text
    );
""")
connection.commit()
```

The triple quotes let the Python string span several lines. The SQL inside is the same kind of table definition we entered at the SQLite prompt.

There are two languages here. Python evaluates the method call and passes a string. SQLite interprets the contents of that string as SQL. A valid Python string can still contain invalid SQL.

Inspecting the Schema from Python

In the interactive shell, `.tables` listed tables. The script instead queries the database's schema information:

```
cursor = connection.execute(
    """
    select name from sqlite_master
    where type = 'table'
    order by name
    """
)
list_of_tables = [item[0] for item in cursor.fetchall()]
print("the tables:")
pprint(list_of_tables)
```

`sqlite_master` is a name for SQLite's schema table. The query returns one column, `name`. Each returned row is a tuple, so `item[0]` extracts that row's name. The list comprehension collects those names into a Python list.

On a fresh practice database, the first check prints `[]`. After table creation, the list is:

```
['pet', 'sqlite_sequence']
```

`sqlite_sequence` supports the table's automatic-identifier behavior. A database used in an earlier run may already contain that internal table, even after `pet` is dropped. The first list need not be empty in every run.

Dot commands are features of the interactive shell. We do not pass `.tables` to `connection.execute()` and expect it to work as SQL.

Supplying Values with Parameters

The script inserts Dorothy this way:

```
connection.execute(
    "insert into pet (name, kind, age, food) values (?, ?, ?, ?)",
    ("Dorothy", "dog", 11, "peanut butter"),
)
```

The first argument describes the SQL operation. The second supplies its values. Each `?` corresponds to one item in the tuple, in order.

Python variables can supply those values just as easily. This variation is equivalent:

```
name = "Dorothy"
kind = "dog"
age = 11
food = "peanut butter"

connection.execute(
    "insert into pet (name, kind, age, food) values (?, ?, ?, ?)",
    (name, kind, age, food),
)
```

Parameter binding keeps a value separate from SQL syntax. A pet named `O'Malley` contains an apostrophe, but the program can pass that name as a value without constructing SQL quotation marks around it. Use parameters for input values rather than inserting them into SQL with string formatting.

The placeholders represent values, not table or column names. The table name `pet` remains part of the SQL statement.

Committing Changes

The script inserts Whiskers next, then commits:

```
connection.execute(
    "insert into pet (name, kind, age, food) values (?, ?, ?, ?)",
    ("Whiskers", "hamster", 1, "hamster chow"),
)

connection.commit()
```

Under the transaction settings used by this script, the two inserts belong to a pending transaction until the commit. A **transaction** groups database changes so they can be accepted together or rolled back. We will examine larger transaction decisions later; here we need to distinguish executing a change from committing it.

`commit()` applies to the connection's pending transaction, not merely the most recent statement. Both inserts precede this call.

A query through the same connection can see its own uncommitted changes. Seeing a row in a query is therefore not, by itself, evidence that another process will find it after this program ends. Explicit commits make the intended saving points visible in this example. Transaction behavior depends on connection settings, so this sequence should not be treated as a rule for every database connection.

Cursors and Returned Rows

`connection.execute()` returns a cursor. For a query, the cursor lets us retrieve its result rows.

The script first fetches one row:

```
cursor = connection.execute("select * from pet")
row = cursor.fetchone()
pprint(row)
```

On the tested run, this printed:

```
(1, 'Dorothy', 'dog', 11, 'peanut butter')
```

This is a Python tuple. Its values follow the selected columns: identifier, name, kind, age, and food. For code that depends on a particular order of rows, include `ORDER BY`; this query does not specify one.

The script executes the query again before fetching all rows:

```
cursor = connection.execute("select * from pet")
rows = cursor.fetchall()
pprint(rows)
```

The observed output was:

```
[(1, 'Dorothy', 'dog', 11, 'peanut butter'),
 (2, 'Whiskers', 'hamster', 1, 'hamster chow')]
```

`fetchall()` gives us a list of the remaining rows. Fetching advances through a result. If we called `fetchone()` and then `fetchall()` on the same cursor, the second call would return only the rows not yet fetched. Re-executing the query is why the script's second result includes Dorothy again.

One Value Still Needs a Sequence

The schema lookup uses one parameter:

```
cursor = connection.execute(
    "select sql from sqlite_master where type='table' and name=?",
    ("pet",),
)
row = cursor.fetchone()
pprint(row[0] if row else "")
```

The comma in `("pet",)` matters. It makes a one-element tuple. `("pet")` is just the string `"pet"` inside parentheses.

There is one placeholder, and the parameter tuple contains one value. The query asks for the stored table definition, so the returned row also contains one value. `row[0]` extracts it.

If the query finds nothing, `fetchone()` returns `None`. The conditional expression prints an empty string in that case instead of attempting to index a missing row. That is a program decision about how to handle an empty result.

When an Operation Fails

The script deliberately tries the wrong table name. With line breaks adjusted, its loop is:

```
print("try with a wrong name")
for i in [1, 2]:
    try:
        connection.execute(
            "insert into petz (name, kind, age, food) values (?, ?, ?, ?)",
            ("Sandy", "cat", 9, "tuna"),
        )
    except sqlite3.Error as e:
        print("Caught sqlite error:", e)
```

The result is:

```
try with a wrong name
Caught sqlite error: no such table: petz
Caught sqlite error: no such table: petz
```

The exception transfers control to `except`, where the program prints the error. Catching it allows execution to continue, but does not make the failed insert succeed.

The loop repeats the same invalid statement twice. Nothing changes between attempts, so retrying cannot fix the misspelled table name.

The following loop uses `pet`, commits the insert, and executes `break` after success. Sandy is inserted once. The `break` exits the loop; it is not a database operation. If a multi-step operation fails, handling the exception and deciding what to roll back are separate responsibilities.

Deleting and Updating Records

After adding Maxwell and Stash, the script removes Stash:

```
connection.execute(
    "delete from pet where name=?", ("Stash",)
)
connection.commit()
print("deletion complete")
```

The tuple supplies the name used by `WHERE`. As before, the condition can match more than one record. Names are not declared unique in this table.

The script also introduces `UPDATE`:

```
connection.execute(
    "update pet set age=12, food='pretzels' where name=?",
    ("Dorothy",),
)
connection.commit()
print("update complete")
```

`SET` names the values to change. `WHERE` selects the records. Dorothy keeps her identifier and kind; her age and food change.

This variation takes all three values from Python:

```
connection.execute(
    "update pet set age=?, food=? where name=?",
    (12, "pretzels", "Dorothy"),
)
connection.commit()
```

An update that matches no rows need not raise an exception. The printed success message means the statement and commit completed, not necessarily that a particular record changed.

Checking the Final State

After the script finishes, a separate process can inspect the database:

```
sqlite3 practice.db -header -column \
  "SELECT id, name, kind, age, food FROM pet ORDER BY id;"
```

The expected records are:

id	name	kind	age	food
1	Dorothy	dog	12	pretzels
2	Whiskers	hamster	1	hamster chow
3	Sandy	cat	9	tuna
4	Maxwell	cat	11	tuna

Stash was inserted and then deleted. The failed inserts into `petz` added nothing. Whiskers is age `1` in this script, rather than `11` as in the earlier terminal transcript.

The original script does not explicitly close its connection. For a program with more work to do after the database operations, close the connection when finished. This additional pattern makes cleanup explicit:

```
connection = sqlite3.connect("practice.db")
try:
    cursor = connection.execute("SELECT name FROM pet ORDER BY name")
    pprint(cursor.fetchall())
finally:
    connection.close()
```

Closing a connection is not a substitute for committing pending changes.

The Application and the Stored Record

The returned tuple is an in-memory representation of stored values. Replacing a Python variable does not update the record automatically. The program needs an SQL operation to change the stored state and a commit at the appropriate point.

That is our first direct connection between Python objects and persistence. For now, the script spells out every operation. The web application will connect those operations to user actions; the database-abstraction topic will collect them behind functions.

Practice

Use a copy of the script and a separate practice database.

1. Run the example with `--db practice.db`. Query the result from the shell and compare it with the final-state table.
2. Add a parameterized insert for a pet named `O'Malley`. Verify the complete name is stored.
3. Query `name` and `age` in that order. Print the returned tuple, then print each value separately.
4. Query a name that is not present. Handle the `None` result from `fetchone()` explicitly.
5. Execute an ordered query, fetch one row, then fetch the rest from the same cursor. Compare this with re-executing the query before fetching all rows.
6. Change Dorothy's age and food using parameters for both new values. Confirm the change from a separate connection after committing.
7. Add explicit connection cleanup to the copied script.

Questions for Thought and Study

1. Which parts of `connection.execute(sql, values)` does Python interpret, and which part does SQLite interpret?
2. Why does `("pet",)` differ from `("pet")`?
3. How does fetching a result differ from executing a query?
4. Why can a row be visible through a connection before its changes are committed?

5. What does catching an exception accomplish, and what does it leave unresolved?
6. Why does the script's startup behavior make it unsuitable for preserving a user's existing pet records?
7. Why does a completed `UPDATE` not necessarily mean a record changed?

Further Reading and Practice

- **Python sqlite3 tutorial and reference:** Connections, parameters, cursors, and transaction behavior.
  https://docs.python.org/3/library/sqlite3.html
- **Python command-line arguments:** The `argparse` module used for `--db`.
  https://docs.python.org/3/library/argparse.html
- **SQLite language reference:** Definitions of the SQL statements used here.
  https://www.sqlite.org/lang.html
- **Python background:** An overview of the language.
  https://en.wikipedia.org/wiki/Python_(programming_language)
- **SQL tutorial and exercises:** More practice with queries and updates.
  https://www.w3schools.com/sql/
  https://www.w3schools.com/sql/sql_exercises.asp